ENGR 102 Summer Bridge
Day 8 Instructor Key

Debugging, Boundary Tests, and Complete Repair Evidence

20 points • approximately 20-25 minutes

Name		UIN		Section	

Boolean-operator convention. Python operators appear as `and`, `or`, and `not`. Their lowercase spelling is required in code.

Show intermediate state for trace credit. Program credit requires one coherent solution. Unless a prompt says otherwise, give exact Python 3 output.

Question 1. Diagnose an inclusive-boundary defect. 5 points

```
temperature = 35
ready = temperature > 15 and temperature < 35
print(ready)
```

The requirement includes 15 and 35. Give actual result, cause, corrected assignment, and two tests that expose the defect.

Answer and explanation

The actual result at 35 is False because temperature < 35 excludes equality. Correct with ready = 15 <= temperature <= 35. Tests 15 -> True and 35 -> True expose both strict-boundary defects; 14.9 and 35.1 should be False.

Question 2. Build a minimum protective boundary set. 5 points

```
reading = 10
low = 10
high = 20

if reading < low:
    status = "below"
elif reading > high:
    status = "above"
else:
    status = "within"

print(status)
```

Give exact supplied output, then specify four tests that protect both inclusive boundaries and their immediate outside cases.

Answer and explanation

The supplied output is within. A strong four-test set is 9.9 -> below, 10 -> within, 20 -> within, and 20.1 -> above. Each expected result targets a distinct comparison boundary.

Question 3. Program construction — 10 points

Write one complete Python program that satisfies every requirement.

- Read reading, low, and high as floats.
- Print invalid when low is greater than high.
- Otherwise print Result: below, Result: within, or Result: above. Equality at either limit is within.
- Use one mutually exclusive chain and provide four boundary tests with expected output.

Answer and explanation

```
reading = float(input())
low = float(input())
high = float(input())

if low > high:
    print("invalid")
elif reading < low:
    print("Result: below")
elif reading > high:
    print("Result: above")
else:
    print("Result: within")
```

The limit-order guard comes first. The below and above branches use strict comparisons, leaving equality at either boundary for within.

For low 10 and high 20, use 9.9 -> Result: below, 10 -> Result: within, 20 -> Result: within, and 20.1 -> Result: above. A reversed pair such as low 20 and high 10 must print invalid.

Scoring guidance

2 input/limit validation, 3 correct ordered classifier, 2 inclusive-boundary behavior, 1 exact output, 2 four-case test evidence.